

第一章 计算机系统概述

1. 计算机发展概述

世界上第一台通用电子数字计算机 ENIAC 使用电子管作为其核心电子器件。

2. 摩尔定律

集成电路上的晶体管数量大约每 18 个月翻一番。

3. 计算机系统的组成

计算机系统由硬件系统和软件系统两大部分组成，其中硬件是计算机的物理基础，软件是运行在硬件之上的程序和数据。

计算机的外围设备是指除了 CPU 和内存以外的其他设备，包括输入设备、输出设备和外存等。

CPU 由运算器和控制器组成：运算器负责算术运算和逻辑运算；控制器负责指令控制与协调工作；计算机中负责算术与逻辑运算的核心部件是 ALU（算术逻辑单元）。

4. 冯·诺依曼计算机的核心思想

- 1) 采用二进制表示数据和指令；
- 2) 程序存储；
- 3) 程序控制；
- 4) 计算机由运算器、控制器、存储器、输入设备、输出设备五大部件组成。

5. 汇编语言和机器语言

汇编语言程序需要经过汇编程序翻译为机器码后才能由硬件执行。

机器语言是最低层计算机语言，用它编写的程序，可以由计算机硬件直接识别。

6. 总线

按照总线上传输的信息种类不同，总线可以分成数据总线、地址总线和控制总线，分别用于传输数据信息、地址信息和控制信号。

7. 计算机系统的性能评价

非时间指标：机器字长、总线宽度、主存容量、存储带宽、CPU 内核数。

时间指标：主频、周期、外频、倍频、CPI、IPC、MIPS、MFLOPS、CPU 执行时间。

其中：

时钟周期 T：时钟周期是计算机中**最基本、最小的时间单位**。在一个时钟周期内，CPU 仅完成一个最基本的动作。

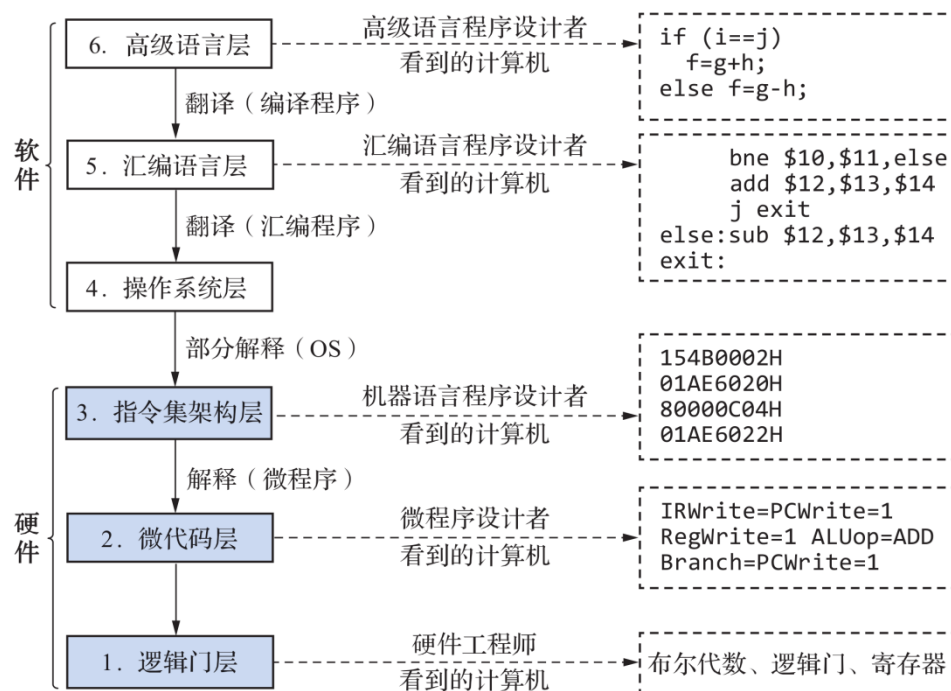
CPI：执行每条指令所需要的平均时钟周期数。

IPC：每个时钟周期 CPU 执行指令的条数，是 CPI 的倒数。

MIPS：每秒执行多少百万条指令。计算公式：MIPS=f / CPI

MFLOPS：每秒执行多少百万条浮点操作。

8. 计算机的层次结构的 6 个层次（简答题）



第二章 数据与信息表示

1. 进制转换

十进制整数转换二进制采用“除 2 取余法”，按权展开（推荐）；

十进制小数转换二进制采用“乘 2 取整法”。

例如：26₁₀=00011010₂、37₁₀=00100101₂、18.75₁₀=10010.11₂

2. 补码

正数的补码等于原码；负数的补码等于其反码加 1。补码和移码只有一个机器 0；

二进制补码表示中，零的表示形式是唯一的。

在一个 n 位二进制数的机器中，补码表示数的范围从 -2^{n-1} 到 $2^{n-1}-1$ ，其中 -2^{n-1} 的补码为 $100\cdots 0000$ ($n-1$ 个 0)。(计算)

3. 非数值数据的表示

ASCII 码主要用于英文字符和常用符号的编码。汉字编码通常采用 GB2312、GBK 或 Unicode。

4. 浮点数的范围和精度

浮点数的取值范围由阶码的位数决定，而精度由尾数的位数决定，阶码越长表示范围越大，尾数越长精度越高。

不是所有的浮点数都能在计算机中精确表示，会出现无限循环小数，例如十进制小数 0.1 转换为二进制时会出现无限循环。

5. 码距

任意两个合法编码间不同的二进制位个数成为码距。码距越大，抗干扰能力、纠错能力越强，数据冗余越大，编码效率越低。

6. 数据信息的校验

1. 奇偶校验码只能检错，不能纠错。
2. 海明码能够实现一位纠错、两位检错。
3. CRC 校验码主要用于数据通信中的检错。

第三章 运算方法与运算器

1. 运算器

算数逻辑运算单元 (ALU)：对顶点数据进行加工处理的纯组合逻辑电路，主要功能包括算数运算和逻辑运算，也常作为数据传送通路。

2. 加减法运算电路

计算机中的减法通常通过补码加法实现： $X-Y=X+(-Y)$ ，在可控加减法电路中，

当 $\text{Sub}=1$ 时，送入加法器的是减数的补码。

3. 原码乘法

原码一位乘法中，符号位不需要参与运算，符号位单独处理，运算时只对数值部分进行乘法运算。每次部分积累加后逻辑右移一位，逻辑右移最高位补 0。

4. 补码一位乘法

乘数采用单符号位，符号位和数据共同参与运算，乘数后面补 0，每次部分积累加后算术右移一位（最后一次不右移），算术右移最高位补符号位，右移前符号位为 1，则补 1，符号位为 0 则补 0。

5. 浮点数的运算规则

浮点数加减法包括五个步骤：①对阶，小阶对大阶；②尾数运算，对尾数进行加减；③运算结果规格化处理，使尾数满足规格化要求；④舍入处理；⑤判断结果是否发生阶码溢出。

浮点数加法中对阶时，小阶向大阶看齐，小阶的尾数需要右移，使阶码与大阶相等。

6. 原码不恢复余数除法

第一次做减法。余数为正时，商上 1，余数左移一位，减去除数。余数为负时，商上 0，余数左移一位，加上除数。若最后一次运算上商时余数小于 0，则需要通过加上除数将余数恢复成正数。

7. 溢出检测方法

- (1) 根据操作数和运算结果的符号位是否一致进行检测。
- (2) 根据运算过程中最高数据位的进位与符号位的进位是否一致进行检测。
- (3) 利用变形补码的符号位进行检测。

第四章 存储系统

1. 按存取方式分类

随机存储器：存取任何存储单元所需时间相同，与位置无关，可直接“跳转”访问。如内存。

顺序存储器：必须按物理顺序依次查找，存取时间取决于位置，不能直接跳

过。如磁带。

直接存储器：不必经过顺序搜索就能在存储器中直接存取信息的存储器。

2. 按功能和存储速度分类

寄存器存储器、高速缓冲存储器、主存储器、外存储器。

3. 存储器分类

SRAM 速度快、不需要刷新，通常用于 Cache。

DRAM 集成度高、需要周期刷新，通常用于主存。

4. ROM 分类

PROM 属于一次可编程只读存储器。

EPROM 采用紫外线擦除。

EEPROM 采用电擦除。

Flash 是 EEPROM 的改进型。

5. 存储元

存储元是存储器中的最小存储单位，其作用是存储一位二进制信息，若干存储元组成一个存储单元，若干存储单元组成一个存储器。

6. 存取时间

存取时间是启动一次存储器操作（读或写分别对应取与存）到该操作完成所经历的时间。

7. 存取周期

存取周期是连续启动两次访问操作之间的最短时间间隔。

8. 存储器带宽

存储器带宽指单位时间内存储器所能传输的信息量。

9. 双译码结构

采用双译码结构的主要目的是降低存储器芯片的引脚数，双译码结构将地址译码分为行译码和列译码，减少了芯片对外引脚的数量。

10. 地址映射

(1) 全相联：各主存块能映射到 cache 的任意数据块，cache 利用率高，冲突少。

(2) 直接相联：各主存块只能映射到 cache 中的固定块（行）。在地址映射中，直接相联映射中每一个主存块地址只能映射到 cache 中固定的行。

设某 cache 共有 m 块，采用直接相联映射，主存共 n 块，则主存块号 i 对应的 cache 块号为 $i = n \bmod m$ （取模），计算方式为主存块号对 cache 块数取模

(3) 组相联：各主存块只能映射到 cache 固定组中的任意块。兼顾了直接相联和全相联的优点；如 cache 被分为了两组，则叫二路组相联，在命中率与硬件成本之间取得折中。

11. 存储器扩展

当存储芯片的数据位宽小于 CPU 数据总线位宽时，需要采用位扩展（字长扩展），需要芯片数：CPU 总位宽 / 芯片位宽。

当存储容量不足时，需要采用字扩展，需要芯片数：CPU 总容量 / 单片芯片容量。

两者都不足时，采用字位同时扩展，一般先位扩展，再字扩展。

某 SRAM 芯片的存储容量为 $128K \times 8$ 位，则该芯片的地址线为 17 条，因为 $128K$ 等于 2 的 17 次方。

12. DRAM 刷新

刷新：定期补充电荷以避免电荷泄露引起的信息丢失。

刷新周期：存储器两次完整刷新之间的时间间隔。

常见的刷新方式有：集中式刷新、分散式刷新、异步式刷新

13. cache

cache 位于主存和 CPU 之间，解决主存和 CPU 的访问速度不匹配的问题，从而提高主存的访问速度。

cache 和主存之间工作的基本原理：程序访问存在局部性，上层存储器可为下层存储器做缓冲，将最经常使用数据的副本调度到上层。局部性包括：时间局部性和空间局部性。

时间局部性：如果某个数据或指令刚被访问过，那么在不久的将来它很可能

再次被访问，如循环变量反复访问。

空间局部性：如果某个存储单元被访问过，那么其附近的存储单元在不久后也很可能被访问，如顺序访问的数组。

多级 cache 设计中，容量通常 L1 Cache 小于 L2 Cache。

14. Cache 替换算法

FIFO (First In First Out) 算法：按照数据块进入 Cache 的先后顺序进行替换，最先进入 Cache 的数据块最先被淘汰。

LRU (Least Recently Used) 算法：替换最近一段时间内最少被使用的数据块，当数据被命中一次，则计数器值恢复 0，其余数据块计数值加 1，计数器值最大的先被替换掉。

LFU (Least Frequently Used) 算法：替换在一段时间内访问次数最少的数据块，需要记录各数据块的访问次数。

15. 虚拟存储器

虚拟存储器处于“**主存-辅存**”存储层次，通过在主存和辅存之间增加部分软件（操作系统）和必要的硬件，使辅存和主存构成一个有机的整体，从而扩大主存的容量。利用程序访问的局部性原理，通过软硬件结合实现逻辑上“容量大、速度接近主存”的存储系统，从而实现以较小的主存运行较大的程序。

虚拟存储器中，**页表的作用是将虚拟地址转换为物理地址**。

16. 写策略

CPU 对 Cache 执行写操作时，常见的写策略主要有写穿法和写回法。

若 Cache 采用写回法，则 Cache 行换出时，只有当该行被修改过才需写回主存，写回法仅在被修改过的脏数据被替换时才写回主存。

若 Cache 采用写穿法，则直接一步到位写到主存才返回。

第五章 指令系统

1. 指令分类

按照操作数地址数量不同，可分为：三地址指令、双地址指令、单地址指令、零地址指令

2. 变长指令系统

变长指令系统的结构灵活,冗余状态较少,平均指令长度较短,可扩展性好,变长指令系统在 CISC 中较为常见,具有良好的编码效率。

3. 寻址方式

指令寻址: 顺序寻址、跳跃寻址;

操作数寻址: 立即寻址、直接寻址、间接寻址、寄存器寻址、寄存器间接寻址、相对寻址、变址寻址、基址寻址、堆栈寻址。

其中: 间接寻址: 在指令的执行阶段需要两次访问主存。

直接寻址的操作数在主存储器中,操作地址由形式地址字段直接给出,这种方式不需要额外的地址计算。

计算机的五大部件通过总线连接,但数据总线的宽度不能决定 CPU 的寻址能力。

a) 地址获取

顺序寻址是指程序按指令在主存中的存放顺序依次执行。CPU 每执行完一条指令后,由程序计数器 PC 自动形成下一条指令地址,通常表示为 $PC + 1$,其中“1”表示一个指令字长(或一条指令的长度),即 PC 自动指向下一条将要执行的指令。

直接寻址: 指令中的地址字段直接给出操作数在主存中的有效地址(EA),即操作数地址可直接获得。

间接寻址: 指令地址字段给出的不是操作数地址,而是存放操作数地址的存储单元地址,因此需要先访问一次主存取出有效地址,再访问主存取出操作数,通常需要多次访存。

相对寻址: 以程序计数器(PC)内容为基准地址,加上指令中的地址偏移量 A 形成有效地址,其计算公式为: $EA = PC + A$,其中 EA 为有效地址,PC 为程序计数器内容, A 为偏移量。

基址寻址: 以基址寄存器(BR)内容作为基准地址,与指令中的地址偏移量 A 相加得到有效地址,其计算公式为: $EA = BR + A$,其中 BR 为基址寄存器内容。

变址寻址: 在变址寻址中,变址寄存器的内容在程序执行过程中可以改变,有效地址等于变址寄存器内容与形式地址之和,变址寻址通过改变变址寄存器的内容实现对数组等数据结构的灵活访问。以变址寄存器(IX)内容与地址偏移量

A 相加形成有效地址，其计算公式为： $EA=IX+A$ ，其中 IX 为变址寄存器内容。

寄存器间接寻址：操作数地址存放在寄存器中，而操作数本身位于主存中。CPU 先从寄存器中取出有效地址，再访问主存取出操作数。

堆栈寻址：利用堆栈存放操作数，堆栈是一种遵循“先进后出（LIFO）”原则的特殊存储结构。操作通常由堆栈指针（SP）自动管理地址，SP 指向栈顶单元。

4. 操作数寻址中操作数来源

操作数的来源基本上有 3 种情况：①操作数直接来自指令地址字段；②操作数存放在寄存器中，即寄存器操作数；③操作数存放在存储器中，即存储器操作数。

5. 程序计数器 PC

CPU 在执行一条指令的过程中，PC 通常指向下一条将要执行的指令地址。

6. 精减指令系统(RISC)

(1)优先选取使用频率最高的一些简单指令，以及一些很有用但不复杂的指令，避免使用复杂指令。

(2)大多数指令在一个时钟周期内完成。

(3)采用 LOAD/STORE 结构。

(4)采用简单的指令格式和寻址方式，指令长度固定。

(5)固定的指令格式。

(6)面向寄存器的结构。

(7)采用硬布线控制逻辑。

(8)注重编译的优化，力求有效地支持高级语言程序。

7. CISC

CISC 特点包括：指令系统复杂；指令格式多；寻址方式丰富；常采用微程序控制。

计算题

1. 补码加减法

$$[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$$

$$[X - Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$$

$$[-Y]_{\text{补}} = [[Y]_{\text{补}}]_{\text{补}}$$

$$[[Y]_{\text{补}}]_{\text{补}} = [Y]_{\text{原}}$$

双符号位溢出检测方法 (变形补码)

$$\begin{array}{r}
 00.10101 \\
 + 00.01000 \\
 \hline
 00.11101
 \end{array}$$

$$\begin{array}{r}
 11.10101 \\
 + 11.11000 \\
 \hline
 11.01101
 \end{array}$$

正溢出

$$\begin{array}{r}
 00.10101 \\
 + 00.11000 \\
 \hline
 01.01101
 \end{array}$$

负溢出

$$\begin{array}{r}
 11.10101 \\
 + 11.11000 \\
 \hline
 10.01101
 \end{array}$$

双符号位最高位永远是正确符号位

$Overflow = f_1 \oplus f_2$

2. 海明码

设海明码 N 位，其中数据位 k 位，校验位 r 位（冗余位）；

满足 $N = k + r \leq 2^r - 1$ ，取满足条件的最小 r ；

2 海明码- (k, r) 码校验分组方法

1. 从下图可以看到 H_1, H_2, H_4, H_8 对应校验位 P ，数据位依次放在 H_3, H_5, H_6 等这些非2的幂次方上（注意： D_1 只能放在 H_3 的位置， D_2 只能放在 H_5 ，后面的数据位依次顺序存放在 H_x ， x 为2的非幂次方）

2. 检错码是对应 $G_4 G_3 G_2 G_1$ 码，所以当对应位置的检错码值为1时，说明此 H_x 对应的值影响该检错码的值，因此在对应的检错码打勾，直到根据所有检错码的值做完打勾标记后，可以得到关于每一个检错码的分组，如图右边公式所示，则

$$G_1 = P_1 \oplus D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7$$

$P_1 = D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7$ ，计算海明码要计算出对应的校验位的值，再根据 $H_1 \dots H_{12}$ 的顺序写出海明码

	H ₁	H ₂	H ₃	H ₄	H ₅	H ₆	H ₇	H ₈	H ₉	H ₁₀	H ₁₁	H ₁₂
检错码	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100
对应位	P ₁	P ₂	D ₁	P ₃	D ₂	D ₃	D ₄	P ₄	D ₅	D ₆	D ₇	D ₈
G ₁ 组	✓		✓		✓		✓		✓		✓	
G ₂ 组		✓	✓			✓	✓			✓	✓	
G ₃ 组				✓	✓	✓	✓					✓
G ₄ 组								✓	✓	✓	✓	✓

$$G_1(P_1, D_1, D_2, D_4, D_5, D_7),$$

$$G_2(P_2, D_1, D_3, D_4, D_6, D_7)$$

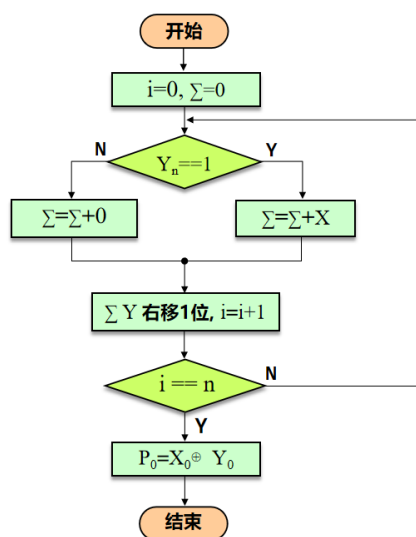
$$G_3(P_3, D_2, D_3, D_4, D_8)$$

$$G_4(P_4, D_5, D_6, D_8)$$

3. 原码一位乘

原码乘法算法流程图

- 作完加法，一定移位
- n次加法
- 符号位单独计算，异或运算

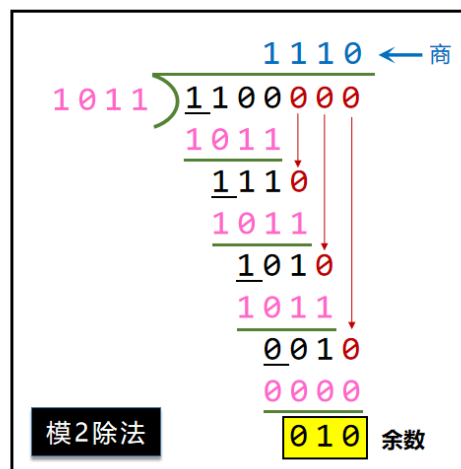


4. CRC 循环冗余校验

编码规则：编码可被生成多项式整除

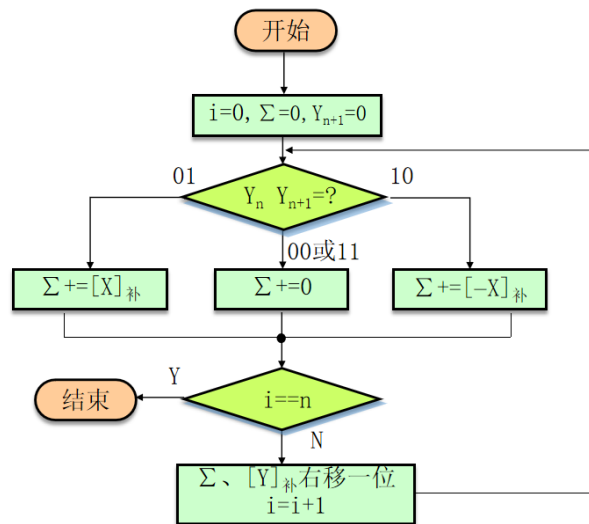
模2运算规则

- 加法：按位加不考虑进位
- 减法：按位减不考虑借位
 - 异或运算，不考虑进位
- 乘法：部分积之和按模2加法计算
- 除法：余数首位为1，商上1，否则为0
 - $1100000 \div 1011$
 - $1100000 = 1011 * 1110 + 010$



5. 补码一位乘

补码一位乘法流程图 (booth一位乘法)



■ $n+1$ 次加法

■ n 次移位

■ 符号位参与运算

■ 不需单独计算符号位